

Invest-Tracker

Placement Interview Guide



Table of Contents

1. [Project Pitch Scripts](#)
 2. [Tech Stack at a Glance](#)
 3. [Project-Specific Interview Q&A](#)
 4. [React Q&A](#)
 5. [Redux Toolkit Q&A](#)
 6. [JavaScript / ES6+ Q&A](#)
 7. [Node.js & Express Q&A](#)
 8. [REST API Design Q&A](#)
 9. [Supabase & PostgreSQL Q&A](#)
 10. [Authentication & JWT Q&A](#)
 11. [Vite & Build Tools Q&A](#)
 12. [TailwindCSS Q&A](#)
 13. [Deployment & Vercel Q&A](#)
 14. [System Design Q&A](#)
-

1. Project Pitch Scripts

30-Second Elevator Pitch

"I built **Invest-Tracker**, a full-stack investment portfolio management app. Users can track their stock holdings and precious metals like gold and silver, see live prices, view market news, and analyze their portfolio performance — all in one place. The frontend is built with **React and Vite**, the backend is a **Node.js/Express REST API**, and data is stored in **Supabase** which is a PostgreSQL-based cloud database. The whole thing is deployed on **Vercel** — two separate deployments for frontend and backend."



2-Minute Interview Answer (for "Tell me about your project")

"I built a project called **Invest-Tracker**, which is a full-stack investment portfolio tracking application — think of it like a personal finance dashboard where you can monitor all your investments in one place.

On the **frontend**, I used **React 18 with Vite** as the build tool, **TailwindCSS** for styling, and **Redux Toolkit** for state management. I built 9 pages — a landing page, a dashboard, portfolio management, stock detail pages, a news feed, a market calendar, and profile/settings pages. I used **Recharts** for data visualization and **Framer Motion** for animations.

On the **backend**, I built a **RESTful API** using **Node.js and Express**. It has three main route modules — stocks, assets (for gold and silver), and news. It integrates with three external APIs: **Finnhub** for live stock prices, **MetalPriceAPI** for gold and silver prices, and **Marketaux** for financial news.

For the **database**, I used **Supabase**, which is a PostgreSQL-based backend-as-a-service. I designed two tables — `stocks` and `assets` — with **Row Level Security** so users can only access their own data. Authentication is handled entirely by Supabase Auth using **JWT tokens**, which I verify on the backend using a custom middleware.

The project is **deployed on Vercel** as two separate projects from the same GitHub monorepo — the frontend as a Vite static site and the backend as Node.js serverless functions.

The main challenges I solved were configuring CORS dynamically using environment variables, guarding the `app.listen()` call so it works in both local and serverless environments, and handling cases where the Finnhub API returns null to prevent crashes."

5-Minute Deep Dive (for technical rounds)

"Let me walk you through the architecture first, then the key technical decisions.

Architecture Overview: It's a monorepo with two separate deployments. The frontend lives in `frontend/` and the backend in `backend/`. In production, they're deployed as two independent Vercel projects that communicate over HTTPS — the frontend calls the backend's REST API endpoints.

Frontend Architecture: The entry point is `main.jsx`, which renders `App.jsx`. App sets up `React Router` for client-side routing and wraps everything in an `AuthProvider` context. I have 9 pages, all except the homepage are protected routes — they check if the user is authenticated before rendering.

The `AuthContext` handles all auth operations — signup, login, logout, password reset — by calling Supabase Auth directly from the frontend. The Supabase session gives us a JWT token.

For API calls, I created a centralized `api.js` service using Axios. It has an **interceptor** that automatically attaches the JWT Bearer token to every outgoing request and handles 401 errors by redirecting to login. This way every component just calls `api.get('/stocks')` without worrying about auth headers.

Backend Architecture: The backend is Express with ES Modules. I have three route files — `stocks.js`, `assets.js`, `news.js` — mounted at `/api/stocks`, `/api/assets`, `/api/news` in `server.js`.

I have a custom `authenticateUser` middleware that extracts the Bearer token from the Authorization header and verifies it using the Supabase `getUser()` API. If valid, it attaches `req.user` to the request so routes can use `req.user.id` to scope database queries to that user.

Database Design: Two tables. `stocks` stores user stock holdings with fields for ticker, shares, buy price, current price, target price, and a `is_in_watchlist` flag. `assets` stores precious metal holdings with grams and price fields. Both tables have `user_id` foreign key and Row Level Security policies so the service role key on the backend has full access while the anon key is restricted.

Key Technical Decisions:

1. *Why Supabase over a traditional DB?* It gave me auth, database, and real-time capabilities in one service, which was perfect for a solo project.
2. *Why serverless on Vercel?* Zero server management, free tier, auto-scaling. The `app.listen()` call is guarded with an env check so it only fires locally.
3. *CORS configuration:* Instead of hardcoding the frontend URL, I made it dynamic via a `FRONTEND_URL` environment variable. This way I can deploy to any domain without changing code.

4. *Fallback for Finnhub failures:* The stock price API sometimes returns null. Instead of crashing, I fall back to the user's buy price as the initial current price, so the stock still gets saved.

The whole thing is on GitHub as a monorepo and CI/CD is through Vercel's GitHub integration — every push to main auto-deploys."

2. Tech Stack at a Glance

Frontend

TECHNOLOGY	VERSION	PURPOSE
React	18	UI library — component-based architecture
Vite	6	Build tool & dev server (replaces Create React App)
React Router DOM	v7	Client-side routing
TailwindCSS	v3	Utility-first CSS framework
Redux Toolkit	Latest	Global state management
Axios	Latest	HTTP client with interceptors
Recharts	Latest	Charts and data visualization
Framer Motion / Motion	v12	Animations and transitions
Radix UI	Latest	Accessible headless UI components
Headless UI	Latest	Accessible modal/dialog components
Supabase JS	v2	Auth and direct DB access from frontend
Lucide React	Latest	Icon library
react-hot-toast	Latest	Toast notifications

TECHNOLOGY	VERSION	PURPOSE
date-fns	Latest	Date utility functions

Backend

TECHNOLOGY	VERSION	PURPOSE
Node.js	Latest LTS	JavaScript runtime
Express	v4	Web framework / REST API
Supabase JS	v2	PostgreSQL client
Axios	Latest	HTTP client (for external APIs)
dotenv	Latest	Environment variable management
cors	Latest	Cross-Origin Resource Sharing
nodemon	Dev only	Auto-restart on file changes

External APIs

API	PURPOSE	FREE TIER
Finnhub	Live stock quotes + search	60 calls/min
MetalPriceAPI	Gold (XAU) & Silver (XAG) prices	100 calls/month
Marketaux	Financial news articles	100 calls/day

Infrastructure

TECHNOLOGY	PURPOSE
Supabase	PostgreSQL DB + Auth + Row Level Security

TECHNOLOGY	PURPOSE
Vercel	Frontend + Backend hosting (serverless)
GitHub	Source control + CI/CD trigger

3. Project-Specific Interview Q&A

Q: What is Invest-Tracker and why did you build it?

It's a personal portfolio tracker that lets users monitor stocks and precious metals. I built it to solve a real problem — tracking investments across different asset classes (stocks + gold/silver) in one unified dashboard. It also gave me hands-on experience with full-stack development, external API integration, authentication, and cloud deployment.

Q: Walk me through the architecture of your project.

It's a monorepo with a React frontend (Vite), a Node.js/Express REST API backend, and Supabase as the database and auth provider. They're deployed as two separate Vercel projects. The frontend communicates with the backend over HTTPS REST APIs, and both communicate with Supabase independently — the frontend for auth sessions, the backend for data CRUD operations.

Q: How does authentication work in your project?

Supabase Auth handles all authentication. When a user logs in, Supabase returns a JWT session token. The frontend stores this in memory (Supabase client manages it). My `api.js` has an Axios request interceptor that reads the current session and attaches `Authorization: Bearer <token>` to every API call. On the backend, the `authenticateUser` middleware extracts this token and calls `supabase.auth.getUser(token)` to verify it. If valid, `req.user` is populated and the route proceeds.

Q: How do you ensure users only see their own data?

Two layers. First, at the API level — all protected routes include `.eq('user_id', req.user.id)` in the Supabase query, so even if someone passes a different ID in the request body, the query is always scoped to the verified user. Second, at the database level — I enabled **Row Level Security (RLS)** on both tables with policies like `USING (auth.uid() = user_id)`. This means even a direct database query can't access another user's data.

Q: What challenges did you face and how did you solve them?

Three main challenges: 1. **Serverless deployment** — `app.listen()` crashes in Vercel serverless. Fixed by guarding it with `if (process.env.VERCEL !== '1')`. 2. **CORS in production** — Hardcoded origins break when you change the frontend URL. Fixed by making it dynamic via `FRONTEND_URL` env var. 3. **Null API responses** — Finnhub sometimes returns null if rate-limited. Fixed by using optional chaining (`quote?.c`) and falling back to `buy_price` so the stock still saves successfully.

Q: How does live stock price updating work?

The `stockPriceService.js` has a `startPeriodicUpdates()` function that runs `updateStockPrices()` immediately and then every 5 minutes using `setInterval`. It fetches all stocks from the database, calls Finnhub's `/quote` endpoint for each one, and updates the `current_price` in Supabase. However, in the Vercel serverless environment, `setInterval` doesn't persist between invocations — so this works locally but not in production. In production, the current price is fetched when a stock is added.

Q: Why did you choose Supabase over MongoDB or a traditional SQL database?

Supabase gave me three things in one: a PostgreSQL database, built-in authentication (no need to build auth from scratch), and Row Level Security for data isolation. For a solo project with a tight timeline, this was a huge productivity win. PostgreSQL is also more suitable than MongoDB here because my data is relational — stocks have a foreign key to users, assets have a foreign key to users.

Q: What is the difference between the `stocks` and `assets` tables?

`stocks` stores equity holdings — each row has a `ticker`, `shares`, `buy_price`, `current_price`, `target_price`, and `is_in_watchlist`. Prices come from Finnhub.

`assets` stores precious metal holdings — each row has a `name` (Gold/Silver), `grams`, `buy_price`, and `current_price`. Prices come from MetalPriceAPI, and I convert from troy ounces to grams by dividing by 28.34.

Q: How is your app deployed? Explain the CI/CD pipeline.

The GitHub repo has two deployment targets on Vercel. Each Vercel project is configured with a specific root directory — `frontend/` for the frontend and `backend/` for the backend. Vercel is connected to the GitHub repo, so every push to `main` triggers an automatic redeploy of both projects. Environment variables (API keys, Supabase credentials) are configured in the Vercel dashboard — they're never in the codebase.

Q: What is the purpose of `vercel.json` in the backend?

It tells Vercel how to handle the backend. It specifies that `src/server.js` should be built with `@vercel/node`, and routes all `/api/(.*)` requests to that file. Without it, Vercel doesn't know it's a Node.js application and would fail to deploy.

Q: What are the security measures in your project?

1. **JWT verification** on every protected route via the auth middleware
 2. **Row Level Security** at the database level — users can only access their own rows
 3. **CORS configuration** — only whitelisted origins can call the API
 4. **Environment variables** — all API keys are in env vars, never in code
 5. **Service role key on backend** (full access), **anon key on frontend** (restricted access)
 6. **Input validation** in routes — checking for required fields, parsing floats properly
-

4. React Q&A

Q: What is React and why did you use it?

React is a JavaScript UI library for building component-based user interfaces using a virtual DOM. I chose it for its component reusability, large ecosystem, and because hooks make state and side-effect management clean and readable.

Q: What are React Hooks? Which ones did you use?

Hooks are functions that let functional components use React features like state and lifecycle. I used: - `useState` — for local component state (loading, error, form data, theme) - `useEffect` — for side effects like fetching data on mount, setting up event listeners - `useContext` — to consume the `AuthContext` (via `useAuth()` custom hook) - `useNavigate` — from React Router, for programmatic navigation

Q: What is the difference between `useState` and `useEffect` ?

`useState` stores data that causes a re-render when changed. `useEffect` runs side effects — things that happen after render, like API calls, subscriptions, or DOM manipulation. It runs after every render by default, or only when specified dependencies change.

Q: What is the Context API? How did you use it?

Context API allows sharing state across the component tree without prop drilling. I created `AuthContext` which holds the current `user`, `loading` state, and auth functions (`signIn`, `signUp`, `signOut`, `resetPassword`, `updateProfile`). Any component can call `useAuth()` to access these without props being passed down.

Q: What is prop drilling and how does Context solve it?

Prop drilling is passing data through multiple intermediate components that don't need it, just to reach a deeply nested child. Context solves this by creating a global store accessible from any component in the tree without explicit prop passing.

Q: What is a controlled vs uncontrolled component?

In a **controlled component**, React controls the form input value through state — `value={formData.name}` and `onChange={handleChange}`. The input's value is always in sync with state. In an **uncontrolled component**, the DOM manages the input's value, accessed via `ref`. I used controlled components throughout the forms in Invest-Tracker.

Q: What is `React.memo` and when would you use it?

`React.memo` is a HOC that memoizes a component — it only re-renders if its props change. Useful for expensive child components that receive the same props frequently. I didn't use it explicitly but would apply it to components like `StockCard` if performance profiling showed unnecessary re-renders.

Q: What is the virtual DOM and how does React use it?

The virtual DOM is an in-memory JavaScript representation of the actual DOM. When state changes, React creates a new virtual DOM tree, diffs it against the previous one (reconciliation), and only updates the parts of the real DOM that changed. This is faster than directly manipulating the DOM.

Q: What is the difference between `useEffect` cleanup and not having one?

Cleanup prevents memory leaks. Without cleanup, event listeners, subscriptions, or timers set up in `useEffect` continue running even after the component unmounts. In my `AuthContext`, I return `subscription.unsubscribe()` from `useEffect` to stop listening to auth state changes when the component unmounts.

Q: How do you handle errors in React?

For async errors (API calls), I use try/catch in async functions and set an error state to display to the user. For unexpected render-time errors, React provides **Error Boundaries** — class components with `componentDidCatch`. I handle API errors in the Axios response interceptor too — 401s redirect to login.

Q: What is React Router and how does protected routing work?

React Router is a library for client-side routing — navigating between pages without full page reloads. `ProtectedRoute` is a wrapper component that checks `useAuth()` — if the user is not logged in, it redirects to `/`. Otherwise it renders the wrapped component. All dashboard routes are wrapped in `ProtectedRoute`.

5. Redux Toolkit Q&A

Q: What is Redux and why use it?

Redux is a predictable state management library. It stores the entire app state in one centralized store. Redux Toolkit (RTK) is the official modern way to write Redux — it reduces boilerplate with `createSlice`, `configureStore`, and built-in immer for immutable updates.

Q: What is a slice in Redux Toolkit?

A slice is a collection of reducer logic and actions for a single feature. I have `stockSlice.js` which defines the state shape for stock data and the reducers that update it.

Q: What is the difference between Redux and Context API?

Context API is good for relatively static global data (theme, auth user). Redux is better for complex, frequently changing state that many components read and write — like stock data that gets fetched, filtered, sorted, and updated across multiple pages. Redux also has DevTools for debugging.

Q: What is a reducer?

A reducer is a pure function that takes the current state and an action, and returns a new state. It must not mutate the current state or have side effects. Redux Toolkit's `createSlice` uses Immer under the hood, so you can write "mutating" syntax that is actually converted to immutable updates.

Q: What is the difference between `useSelector` and `useDispatch` ?

`useSelector` reads data from the Redux store. `useDispatch` gives you the dispatch function to send actions that update the store.

6. JavaScript / ES6+ Q&A

Q: What is async/await and how does it work?

`async/await` is syntactic sugar over Promises. An `async` function always returns a Promise. `await` pauses execution inside the function until the awaited Promise resolves, without blocking the main thread. I use it throughout — `const response = await api.get('/stocks')`.

Q: What is the difference between `var` , `let` , and `const` ?

`var` is function-scoped, hoisted, and can be re-declared. `let` is block-scoped, not hoisted to an initializable state, and can be reassigned. `const` is block-scoped and cannot be reassigned (but objects/arrays it points to can be mutated). I always use `const` by default and `let` when reassignment is needed.

Q: What are arrow functions and how are they different from regular functions?

Arrow functions (`=>`) are concise, don't have their own `this` binding (inheriting it from the enclosing scope), and can't be used as constructors. Regular functions have their own `this` context. In React, arrow functions are used heavily in event handlers and callbacks to avoid `this` binding issues.

Q: What is destructuring?

Destructuring extracts values from objects or arrays into separate variables. Example from the project: `js const { name, ticker, shares, buy_price } = req.body; const { data: { session } } = await supabase.auth.getSession();`

Q: What is the spread operator?

`...` copies properties from one object/array into another. Example: `js setFormData(prev => ({ ...prev, [name]: value }));` This creates a new object with all existing `prev` properties, overriding just the one that changed.

Q: What is optional chaining (`?.`)?

It safely accesses nested properties without throwing if an intermediate value is `null` or `undefined` . Returns `undefined` instead of throwing. `js const currentPrice = quote?.c || parsedBuyPrice;`

Q: What is the difference between `==` and `===` ?

`==` does type coercion (e.g. `"1" == 1` is true). `===` checks value AND type (e.g. `"1" === 1` is false). Always use `===` to avoid unexpected behavior.

Q: What are Promises?

A Promise represents a future value — it's either pending, resolved (fulfilled), or rejected. You chain `.then()` for success and `.catch()` for errors. `async/await` is built on top of Promises.

Q: What is closure?

A closure is when an inner function has access to variables of its outer function even after the outer function has returned. Example: every React component's event handler is a closure — it "closes over" state variables like `formData`.

Q: What is event delegation?

Attaching a single event listener to a parent element instead of many listeners on child elements. The event bubbles up and the parent handles it by checking `event.target`. React uses synthetic events which work similarly under the hood.

Q: What is the difference between `map`, `filter`, and `reduce` ?

- `map` — transforms each element, returns new array of same length
 - `filter` — keeps elements matching a condition, returns new array (possibly shorter)
 - `reduce` — accumulates elements into a single value (sum, object, etc.)
-

Q: What is `localStorage` and how did you use it?

`localStorage` persists key-value string data in the browser across sessions. I used it to persist the user's theme preference: `localStorage.setItem('theme', newTheme)` and `localStorage.getItem('theme')`.

7. Node.js & Express Q&A

Q: What is Node.js?

Node.js is a JavaScript runtime built on Chrome's V8 engine that lets JavaScript run on the server. It's non-blocking and event-driven, making it efficient for I/O-heavy tasks like API servers.

Q: What is Express?

Express is a minimal Node.js web framework. It provides routing, middleware support, and request/response handling. I use it to define API routes (`router.get` , `router.post`) and middleware (`app.use(cors())` , `app.use(express.json())`).

Q: What is middleware in Express?

Middleware are functions that run between a request and a response. They have access to `req` , `res` , and `next` . I use middleware for: CORS (`cors()`), JSON parsing (`express.json()`), and my custom `authenticateUser` which verifies JWT before the route handler runs.

Q: What is the purpose of `app.use(express.json())` ?

It parses incoming requests with JSON bodies and populates `req.body` . Without it, `req.body` would be `undefined` for POST/PUT requests.

Q: What is ES Modules vs CommonJS?

CommonJS (the default in Node.js) uses `require()` and `module.exports`. ES Modules (ESM) uses `import` and `export`. The backend uses `"type": "module"` in `package.json` to enable ESM, so I write `import express from 'express'` instead of `require('express')`.

Q: How do you handle errors in Express?

In each route handler with try/catch. In the catch block, I send an appropriate HTTP status code and error message: `res.status(500).json({ error: error.message })`. For uncaught errors, Express has a 4-argument error middleware `(err, req, res, next)`.

Q: What is CORS and why is it needed?

Cross-Origin Resource Sharing — browsers block fetch/XHR requests to a different origin (protocol+domain+port) than the page's origin. The backend must send `Access-Control-Allow-Origin` headers to permit the frontend's origin. I use the `cors` npm package and configure it with a list of allowed origins.

Q: What is the difference between `app.listen()` and a serverless function?

`app.listen()` keeps a long-running process that listens for connections. A serverless function spins up on-demand for each request and shuts down after — no persistent process. Vercel uses serverless, so I guard `app.listen()` to only run locally.

Q: What is `dotenv` and why use it?

`dotenv` loads environment variables from a `.env` file into `process.env`. This keeps secrets (API keys, DB credentials) out of the codebase. In production, these are set directly in the hosting platform (Vercel dashboard).

8. REST API Design Q&A

Q: What is a REST API?

REST (Representational State Transfer) is an architectural style for APIs. It uses HTTP methods to represent operations on resources. It's stateless — each request contains all necessary information.

Q: What are the main HTTP methods and what do they mean?

- **GET** — Read data (no side effects)
- **POST** — Create a new resource
- **PUT** — Update/replace an existing resource
- **PATCH** — Partially update a resource
- **DELETE** — Delete a resource

Q: What HTTP methods do you use in your project and for what?

METHOD	ROUTE	PURPOSE
GET	<code>/api/stocks</code>	Fetch user's stocks
POST	<code>/api/stocks</code>	Add a new stock
PUT	<code>/api/stocks/:id</code>	Update a stock
DELETE	<code>/api/stocks/:id</code>	Delete a stock
GET	<code>/api/stocks/summary</code>	Get portfolio totals
GET	<code>/api/stocks/search?q=</code>	Search stocks
GET	<code>/api/assets</code>	Fetch user's assets
POST	<code>/api/assets</code>	Add a new asset
GET	<code>/api/news/search</code>	Get financial news
GET	<code>/api/health</code>	Health check

Q: What are HTTP status codes? Which ones do you use?

- `200 OK` — Success (GET, PUT)
- `201 Created` — Successfully created (POST)
- `400 Bad Request` — Invalid input from client
- `401 Unauthorized` — Not authenticated
- `403 Forbidden` — Authenticated but not permitted
- `404 Not Found` — Resource doesn't exist
- `500 Internal Server Error` — Server-side failure

Q: What is the difference between query params, path params, and request body?

- **Path params:** Part of the URL path — `/api/stocks/:id` — identifies a specific resource
- **Query params:** Appended after `?` — `/api/stocks/search?q=AAPL` — used for filtering/searching
- **Request body:** JSON payload in POST/PUT requests — used for creating/updating resource data

Q: What is statelessness in REST?

Each HTTP request must contain all information needed to process it. The server doesn't store session state between requests. My API is stateless — the JWT token in each request header carries the user identity.

9. Supabase & PostgreSQL Q&A

Q: What is Supabase?

Supabase is an open-source Firebase alternative built on PostgreSQL. It provides a database, Auth, auto-generated REST APIs, real-time subscriptions, and storage — all as a managed cloud service.

Q: What is PostgreSQL?

PostgreSQL is a powerful open-source relational database. It uses structured tables with defined schemas, supports ACID transactions, foreign keys, joins, and advanced features like JSONB and full-text search.

Q: What is Row Level Security (RLS)?

RLS is a PostgreSQL feature where you define policies on tables that control which rows a user can read/write based on the current user's context. In Supabase, `auth.uid()` returns the logged-in user's ID. My policy `USING (auth.uid() = user_id)` ensures users can only access their own rows — even if someone bypasses the API and queries directly.

Q: What is a primary key? What is a foreign key?

A **primary key** uniquely identifies each row — `id UUID DEFAULT gen_random_uuid()`. A **foreign key** references a primary key in another table — `user_id UUID REFERENCES auth.users(id)` creates a relationship between stocks and users.

Q: What is the difference between SQL and NoSQL?

SQL databases (PostgreSQL) store data in structured tables with predefined schemas, support joins, and guarantee ACID properties. NoSQL databases (MongoDB) store data in flexible documents/collections, are schema-less, and are often horizontally scalable. SQL is better for relational data; NoSQL for unstructured or rapidly changing schemas.

Q: What is a JOIN in SQL?

JOIN combines rows from two tables based on a related column. INNER JOIN returns rows with matches in both tables; LEFT JOIN returns all rows from the left table plus matches from the right. (Not explicitly used in this project since queries are single-table with `user_id` filter.)

Q: How does the Supabase JS client work?

The Supabase client provides a fluent query builder: `js const { data, error } = await supabase .from('stocks') .select('*') .eq('user_id', req.user.id) .gt('shares', 0);` This generates a REST API call to PostgREST (Supabase's auto-generated API layer) which executes the SQL query.

Q: What is the difference between the service role key and the anon key?

The **anon key** is public-safe — it respects RLS policies. The **service role key** bypasses RLS and has full admin access. The frontend uses the anon key (it's in the bundle). The backend uses the service role key (it's in server-side env vars). Never expose the service role key to the frontend.

10. Authentication & JWT Q&A

Q: What is JWT?

JSON Web Token — a compact, URL-safe token format. It has three parts separated by dots: **Header** (algorithm), **Payload** (claims — user ID, expiry, etc.), **Signature** (for verification). It's base64-encoded but not encrypted — the payload is readable, but the signature can't be forged without the secret.

Q: How does JWT authentication work?

1. User logs in → server verifies credentials → returns a JWT
2. Client stores the JWT (in memory / localStorage)
3. Client sends JWT in `Authorization: Bearer <token>` header on subsequent requests
4. Server verifies the signature → extracts user claims → processes the request

Q: What is the difference between authentication and authorization?

Authentication — verifying who you are (login, JWT verification). **Authorization** — what you're allowed to do (RLS policies, ownership checks like `user_id === req.user.id`).

Q: Why is the token not stored in localStorage in Supabase?

Supabase stores the session in `localStorage` by default (in browser environments), but manages it internally. The risk with `localStorage` is XSS attacks — malicious scripts can read it. HTTP-only cookies are more secure but require more setup. For this project, the Supabase default is acceptable.

Q: What happens when a JWT expires?

The server rejects it with a 401. In my Axios interceptor, a 401 response redirects the user to the login page. Supabase also handles token refresh automatically via the auth state change listener.

Q: What is the difference between session-based and token-based auth?

Session-based: Server stores session data and gives client a session ID (cookie). Stateful — server must store sessions. **Token-based (JWT):** Server is stateless — the token contains all user info. The server only needs the secret to verify. Scales better horizontally.

11. Vite & Build Tools Q&A

Q: What is Vite and why use it over Create React App?

Vite is a modern frontend build tool. Unlike CRA which bundles everything with Webpack before serving, Vite serves native ES modules in development — near-instant HMR (hot module replacement) and startup. Production builds use Rollup. It's faster, leaner, and more modern than CRA.

Q: What is HMR (Hot Module Replacement)?

HMR updates changed modules in the browser without a full page refresh, preserving component state. This makes development much faster — you see changes in milliseconds.

Q: What does `vite build` do?

It bundles the app for production using Rollup — tree-shaking unused code, minifying JavaScript and CSS, code-splitting for lazy loading, and outputting optimized static files to `dist/`.

Q: What is tree-shaking?

The process of removing unused code from the final bundle. If you import only `{ useState }` from React, tree-shaking ensures the entire React library isn't bundled — only the parts you use.

Q: What are environment variables in Vite?

Vite exposes env vars prefixed with `VITE_` to the frontend via `import.meta.env`. Example: `import.meta.env.VITE_API_BASE_URL`. Variables without the `VITE_` prefix are not exposed to the browser (kept server-side only).

12. TailwindCSS Q&A

Q: What is TailwindCSS?

TailwindCSS is a utility-first CSS framework. Instead of writing custom CSS classes, you compose styles using pre-built utility classes directly in HTML/JSX —

```
className="flex items-center gap-4 p-6 bg-blue-500 text-white rounded-xl".
```

Q: What are the advantages of TailwindCSS?

- No context switching between HTML and CSS files
- Design system built in (consistent spacing, colors, typography)
- PurgeCSS removes unused classes in production — tiny bundle
- Highly customizable via `tailwind.config.js`
- Dark mode support with `dark:` prefix

Q: How does dark mode work in Tailwind?

By setting `darkMode: 'class'` in `tailwind.config.js`, dark mode classes (`dark:bg-gray-900`) activate when the `dark` class is on `<html>`. I toggle this with `document.documentElement.classList.toggle('dark', theme === 'dark')`.

Q: What is the difference between Tailwind and regular CSS?

Regular CSS requires writing named classes and rules in separate `.css` files. Tailwind uses pre-defined utility classes inline. Tailwind removes the need to name things and eliminates unused CSS automatically in production.

13. Deployment & Vercel Q&A

Q: What is Vercel?

Vercel is a cloud platform for frontend and serverless deployments. It integrates with GitHub for CI/CD — every push auto-deploys. It serves frontend as a CDN-backed static site and backend as serverless functions (AWS Lambda under the hood).

Q: What is a serverless function?

A function that runs on-demand in a stateless, managed container. No server to manage — it spins up for each request, executes, and terminates. Scales automatically, billed per invocation. Vercel uses Node.js serverless functions for the backend.

Q: What is the difference between your frontend and backend deployment on Vercel?

Frontend — Vite builds static HTML/JS/CSS files. Vercel serves them from a CDN (Content Delivery Network) globally. Backend — Vercel detects the `vercel.json` and deploys `src/server.js` as a serverless Node.js function.

Q: What is a CDN?

Content Delivery Network — a distributed network of servers that caches and serves static assets from the server geographically closest to the user. This reduces latency. Vercel automatically serves the frontend via its global CDN.

Q: What is CI/CD?

Continuous Integration / Continuous Deployment. CI means code is automatically tested on every push. CD means code is automatically deployed after passing CI. In my project, every push to `main` on GitHub triggers Vercel to rebuild and redeploy both projects automatically.

Q: How do you manage secrets/API keys in production?

All secrets are stored as environment variables in the Vercel dashboard — never in the codebase. Locally, they're in `.env` files which are in `.gitignore`. This follows the 12-factor app methodology.

14. System Design Q&A

Q: How would you scale this application to handle millions of users?

- **Database:** Move from a single Supabase instance to read replicas + connection pooling
- **Caching:** Add Redis to cache Finnhub API responses (stock prices change every minute, not millisecond) to reduce API calls
- **Rate limiting:** Add rate limiting middleware to prevent API abuse
- **Message queue:** Use a queue (like BullMQ) for the periodic price update job instead of `setInterval`
- **CDN:** Static assets already on CDN via Vercel
- **Horizontal scaling:** Serverless already scales horizontally by default

Q: How would you add real-time price updates?

Instead of polling every 5 minutes, use **WebSockets** or **Server-Sent Events (SSE)** to push price updates to connected clients. Supabase supports real-time subscriptions — I could subscribe to changes in the `stocks` table and push updates to the browser instantly.

Q: What would you improve if you had more time?

1. Real historical price data from Finnhub's candle API (currently simulated)
2. WebSocket-based real-time price streaming
3. Unit tests (Jest for backend, React Testing Library for frontend)
4. A watchlist feature (the `is_in_watchlist` flag already exists in the schema)
5. Portfolio performance charts over time (monthly/yearly returns)
6. Email alerts when a stock hits the target price
7. Multi-currency support

Q: What is the difference between SQL and REST when accessing data?

REST (via Supabase client) is abstracted and works over HTTP with JSON. It's good for CRUD from application code. SQL is the raw query language — more powerful for complex joins, aggregations, and analytics. In my project, I use the Supabase JS client (which generates REST calls internally) rather than raw SQL.

End of Interview Guide